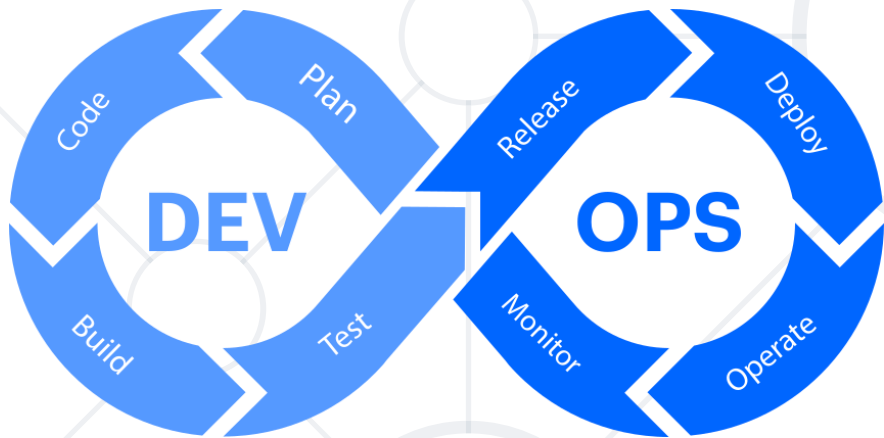


Monitoring & Observability

From Guessing to Knowing



SoftUni Team
Technical Trainers



SoftUni



Software University

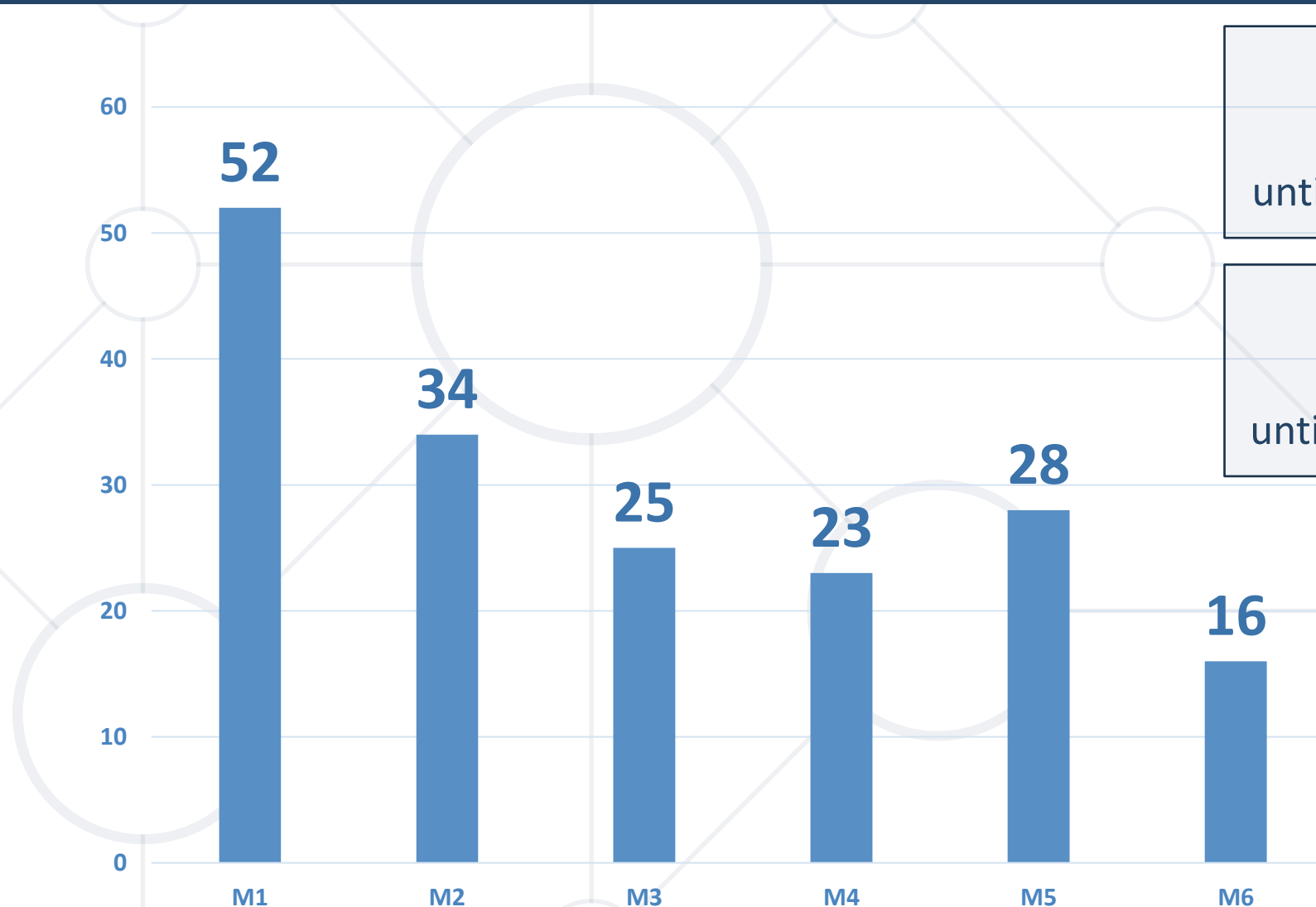
<https://softuni.bg>

sli.do

#DevOps-CI-CD

[facebook.com/groups/
cicdmonitoringjanuary2026](https://facebook.com/groups/cicdmonitoringjanuary2026)

Homework Progress



Solutions for M6
can be submitted
until 23:59 on 04.03.2026

Solutions for M7
can be submitted
until 23:59 on 11.03.2026

THIS MODULE
AND ONE MORE TO GO

Web App Will Be Available

<https://zahariev.pro/q/do2>



Previous Module (M6)

Quick Overview

What We Covered

- 
1. CI/CD vs GitOps
 2. Benefits and Challenges
 3. Tooling Landscape
 4. Implementation

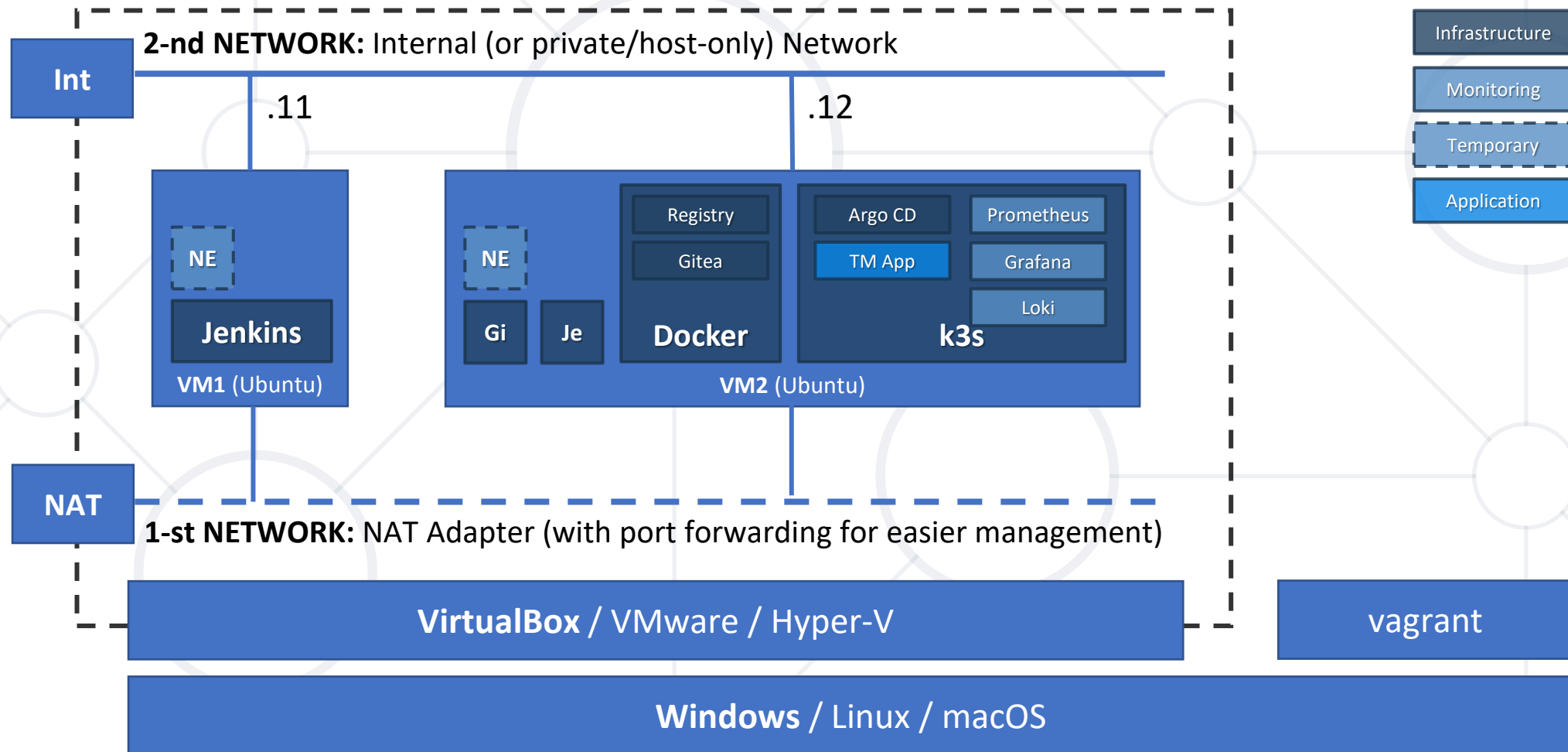


This Module (M7)
Topics and Infrastructure

1. Monitoring vs Observability
2. Monitoring: The Four Golden Signals
3. Observability: The Three Pillars
4. Collecting, Presenting, and Acting on Data



Lab Infrastructure





The Foundation of Visibility

Overview

Monitoring vs. Observability

- **Monitoring**

- Watching a predefined set of metrics to see if the system is healthy
- Focuses on system health using alerting and dashboarding
- For example, "Is the CPU over 80%?"

- **Observability**

- The ability to understand the internal state of a system by looking at the data it produces
- Focuses on internal state/root cause usually for troubleshooting
- For example, "Why is this user experiencing 500 errors?"

The Four Golden Signals

- **Latency**
 - Time it takes to service a request
- **Traffic**
 - Demand placed on the system (e.g., HTTP requests/sec)
- **Errors**
 - The rate of requests that fail
- **Saturation**
 - How "full" a service is (e.g., memory usage, CPU, etc.)

The Three Pillars of Observability

- **Metrics (the What)**
 - Aggregated data (Count/Rate/Gauge)
 - For example, "The endpoint was visited 50 times"
- **Logs (the Why)**
 - Discrete events
 - For example, "User Admin logged in at 10:01 AM"
- **Traces (the Where and How)**
 - The journey of a request across microservices.



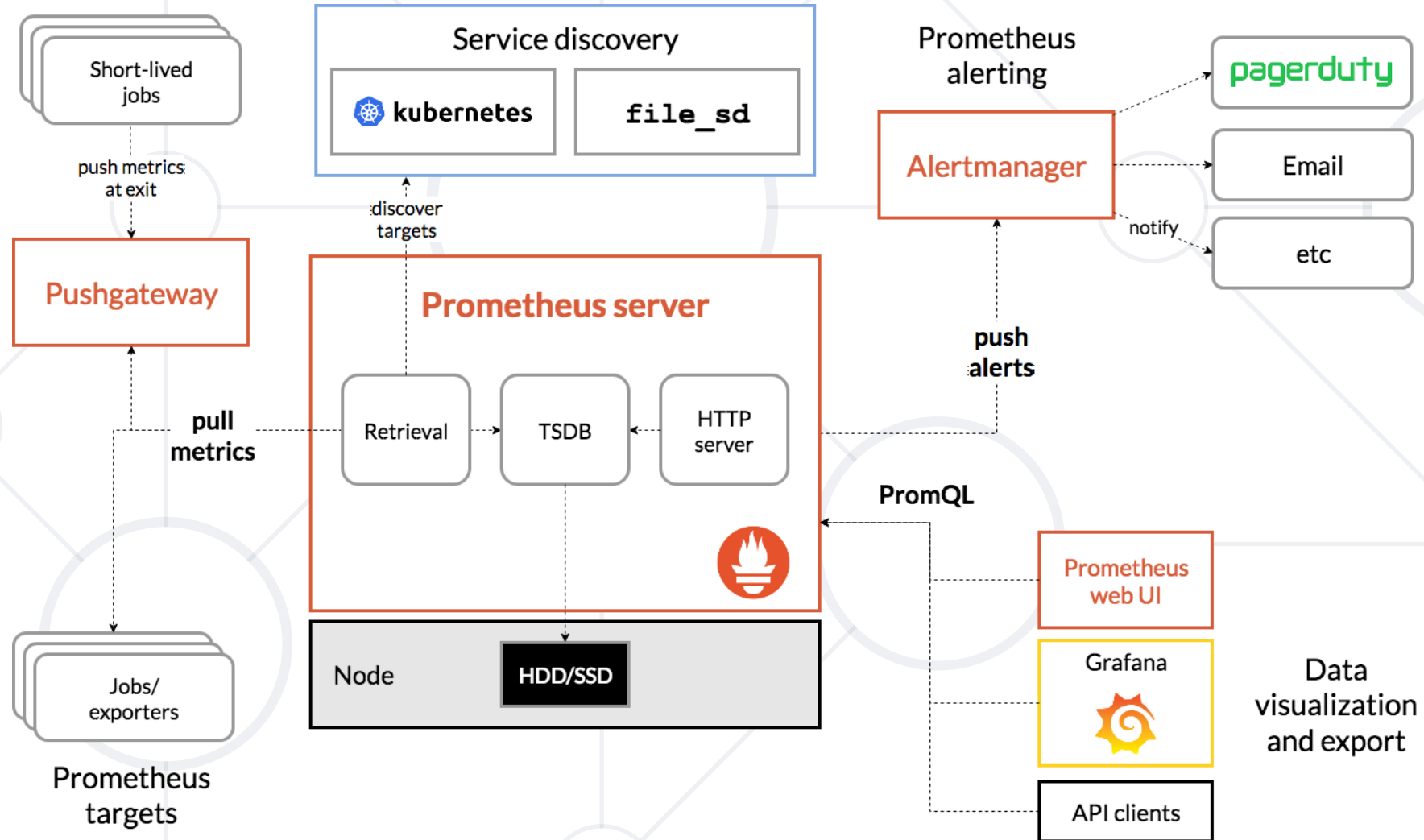
Prometheus

Getting to Know Prometheus

What is Prometheus?

- An open-source toolkit for **monitoring** and **alerting**
- Collects and stores **metrics** as **time series data**
- Additionally, it can store **optional labels** (key-value pairs)
- Offers flexible query language (**PromQL**)
- Data collection happens via **pull model** over HTTP
- Pushing of data is also supported via an **intermediary gateway**
- Suitable for recording pure numeric time series
- Not suitable when we need 100% accuracy (for example, for billing)

Architecture



- Pre-compiled binaries
 - Most popular operating systems are supported
- Build from source
- Using containers
- Using configuration management systems
 - Ansible, Chef, Puppet and Salt
- Different components are available as separate artefacts

- Configured via **command-line flags** and a **configuration file**
- Command-line arguments are used to configure **immutable system parameters** like storage location, amount, etc.
- Configuration file is used to control **scraping** and **rules**
- The configuration file is in **YAML** format
- It can be reloaded (including any rules) by sending
 - **SIGHUP** to the Prometheus process
 - **HTTP POST** request to **/-/reload** endpoint (***--web.enable-lifecycle*** flag)

- Data is stored as **time series**
- Which are **streams of timestamped values** belonging to the same metric and the same set of labeled dimensions
- Every time series is uniquely identified by its **metric name and optional key-value pairs** called labels
- The metric name specifies the general feature of a system that is measured
- Labels enable Prometheus's dimensional data model
- Any given combination of labels for the same metric name identifies a particular dimensional instantiation of that metric

```
api_http_requests_total{method="POST", handler="/messages"}
```

- **Counter** is a cumulative metric that represents a single monotonically increasing counter whose value can only increase or be reset to zero on restart. For example, number of tasks completed, requests served, etc.
- **Gauge** is a metric that represents a single numerical value that can arbitrarily go up and down. For example, temperatures, number of concurrent requests, etc.
- **Histogram** samples observations (like request durations or response sizes) and counts them in configurable buckets. It also provides a sum of all observed values
- **Summary** samples observations (like request durations and response sizes). While it also provides a total count of observations and a sum of all observed values, it calculates configurable quantiles over a sliding time window

- An endpoint that we can scrape is called an **instance**
- Usually, it corresponds to a single process
- A collection of instances is called a **job**
- When target is scraped, two additional labels are added – **job** and **instance**
- Each instance scrape adds sample to a set of system time series
- A **sample** is a single value at a point in time in a time series
- Each sample consists of a float64 value and a millisecond-precision timestamp



PromQL

Introduction

- PromQL = Prometheus Query Language
 - It allows to select and aggregate time series data in real time
 - The result of an expression can either be
 - shown as a graph
 - viewed as tabular data
 - consumed by external systems via the HTTP API
- } in Prometheus's expression browser

- Expressions and sub-expressions evaluate to one of the following
- **Scalar** - a simple numeric floating-point value
- **String** - a simple string value; currently unused
- **Instant vector** - a set of time series containing a single sample for each time series, all sharing the same timestamp
- **Range vector** - a set of time series containing a range of data points over time for each time series

- Instant vector selectors allow the selection of a set of time series and a single sample value for each at a given timestamp (instant)

<code>http_requests_total</code>	<code># all time series with the specified metric name</code>
----------------------------------	---

- Range vector selectors work the same way but for a range
- A time range is appended at the end of the selector
- It specifies how far back in time values should be fetched

<code>http_requests_total[5m]</code>	<code># values for the 5 minutes</code>
--------------------------------------	---

- We can select just some of the time series for a metric with **exact match**

```
http_requests_total{job="web",os="win"}
```

- Other operators include **not equal (!=)**

```
http_requests_total{job="web",method!="POST"}
```

- **regex-match (=~)**

```
http_requests_total{job="web",environment=~"stage|dev|test"}
```

- and **not regex-match (!~)**

```
http_requests_total{job="web",host!~".*dev.*"}
```

- Time durations (ms, s, m, h, d, w, y) are specified like **[5m]** and can be concatenated

```
http_requests_total{job="prometheus"}[1h5m]    # values for the last hour and 5 minutes
```

- Offset modifier** can be used to change the **start point in time**

```
http_requests_total offset 5m                  # value as of 5 minutes ago
```

- @** modifier is used to change the evaluation time for individual instant and range vectors
- Offset** and **@** can be combined. In addition, we can use the **start()** and **end()** functions

```
http_requests_total offset 5m @ 1646937045    # value as of 5 min before 20:30:45  
                                              # on 10.03.2022
```

- **Arithmetic** binary operators (+, -, *, /, %, ^) between scalar/scalar, vector/scalar, and vector/vector value pairs
- **Comparison** binary operators (==, !=, >, <, >=, <=) between scalar/scalar, vector/scalar, and vector/vector value pairs
- **Logical/set** binary operators (**and**, **or**, **unless**) between instant vectors
- **Aggregation** operators (**sum**, **min**, **max**, avg, etc.) on a single instant vector

(1) Trigonometric binary operators are omitted from the list above

(2) Operations between vectors attempt to find a matching element in the right-hand side vector for each entry in the left-hand side. Two types of matching behavior - one-to-one and many-to-one/one-to-many.



Grafana

Getting to Know Grafana

- Grafana is a complete observability stack
- Monitor and analyze metrics, logs and traces
- Query, visualize, alert on and understand our data
- Create, explore, and share beautiful dashboards
- Offered in three variants – **open source**, **cloud**, and **enterprise**
- Includes additional components like **Loki** and **Tempo**

- **Native packages** (for various Linux distributions)
- Windows installation either with **installer** or **binary**
- macOS via **Homebrew** or **standalone binaries**
- Docker image – **Alpine** or **Ubuntu** based
- Supported databases are **SQLite**, **MySQL**, and **PostgreSQL**
- Supported browsers are **Chrome**, **Firefox**, **Safari**, and **Edge**

- Server level settings are spread across three files **defaults.ini**, **grafana.ini**, and **custom.ini**
- We can control the instance and the server
- There is **Grafana CLI** which can be used to automate stuff
- It can be set up for high availability

- Most popular are supported out of the box
- Prometheus, Graphite, OpenTSDB, and InfluxDB
- Loki and Elasticsearch
- MySQL, PostgreSQL, MicrosoftSQL
- Cloud services
- Many enterprise and third-party plugin

- Panels are the building blocks of dashboards
- They connect via query to a data source
- On them we use different visualizations
- Those can be time series, charts, gauges, tables, etc.
- We then assemble panels as a dashboard



Practice: Start Simple
Getting to Know the Tools



Moving to Kubernetes

And Adding Metrics to Applications

Installation Options for Kubernetes

- Via plain manifests or Helm charts
- Individual components or operator-based approach
- Popular charts are
 - Kube-Prometheus-Stack Helm Chart (<https://github.com/prometheus-community/helm-charts/>)
 - Grafana Kubernetes Helm Charts (<https://github.com/grafana/helm-charts>)
 - Grafana Community Kubernetes Helm Charts (<https://github.com/grafana-community/helm-charts>)

- Client libraries implement Prometheus metric types
- They are useful for monitoring applications and services
- Popular languages like Go, Java, Python, etc. are supported
- There are also unofficial third-party client libraries for other languages
- If there is no library for the language we use, or we want to avoid dependencies, we could also implement one of the supported exposition formats directly



Practice: K8s and App Metrics

Move to the Next Level



Visibility End-to-End
Keep an Eye on Everything

- **Gitea**

- Must be enabled in the configuration
- More here: <https://docs.gitea.com/administration/config-cheat-sheet>

- **Jenkins**

- Install the Prometheus metrics plugin
- More here: <https://plugins.jenkins.io/prometheus/>

- **Argo CD**

- Metrics are there, just have to be exposed to Prometheus
- More here: <https://argo-cd.readthedocs.io/en/stable/operator-manual/metrics/>

- Loki is a horizontally scalable, highly available, multi-tenant log aggregation system inspired by Prometheus
- It does not index the contents of the logs, but rather a set of labels for each log stream
- It is designed to be very cost effective and easy to operate
- Uses LogQL as a query language (similar to PromQL)

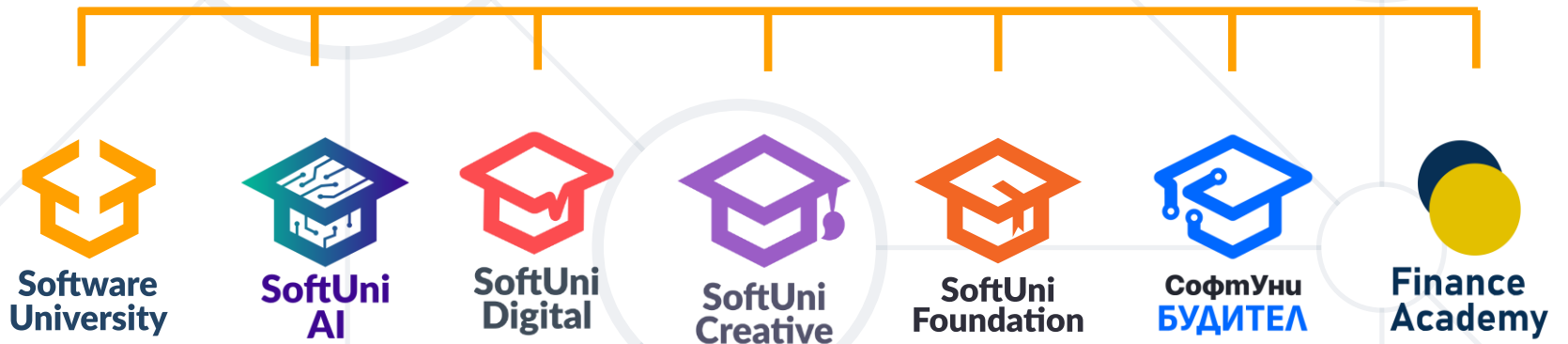


Practice: Visibility End-to-End
Let's Move Even Further

Questions?



SoftUni



- Software University – High-Quality Education, Profession and Job for Software Developers
 - softuni.bg, about.softuni.bg
- Software University Foundation
 - softuni.foundation
- Software University @ Facebook
 - facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

